Least Privilege and Zero Trust

limit the blast radius; assume breach

Watch the 4-min video

In one line: give every identity only the permissions it needs, for only as long as it needs them, and never trust a request because of where it comes from: authenticate, check context and authorize each request, segment the network and encrypt everything.

1 The bug wasn't the problem

An attacker gets into a small email service through a vulnerable library. That service's credentials can read every database and storage bucket in the account, so one small hole becomes a full breach. The real failure was **how much the compromised service was allowed to do**.

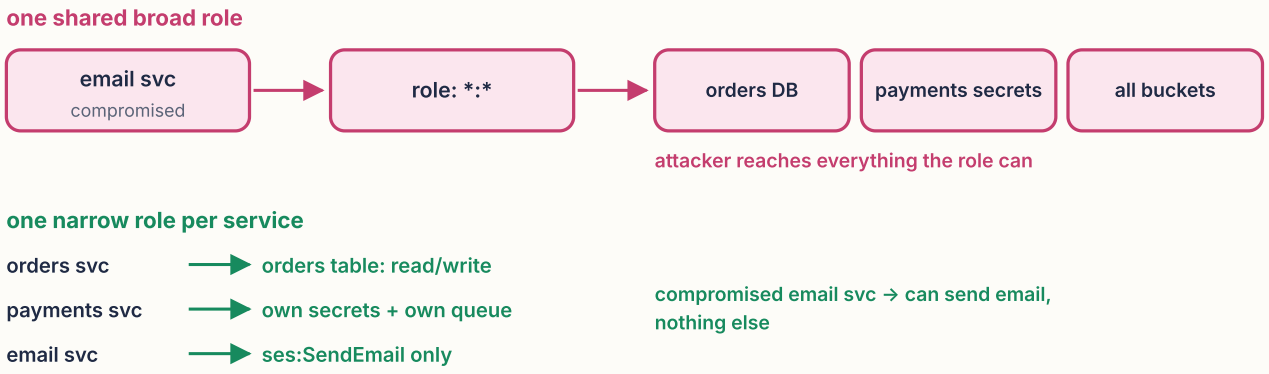


Figure 1. Least privilege doesn't stop the break-in; it limits the blast radius to what that one identity could already do.

2 Least privilege for services

- **One identity per service** (lesson 45), never shared keys, so permissions and audit trails are per service.
- **Narrow policies**: specific actions on specific resources, e.g. `dynamodb:GetItem, PutItem, Query` on `arn:aws:dynamodb:…:table/orders`. A wildcard (`"Action": "*", "Resource": "*"`) is a red flag in every review.
- **Remove unused permissions**: AWS IAM Access Analyzer, GCP IAM Recommender and Entra permissions reports find grants never used in 90 days.
- Apply it to CI/CD too: a pipeline that deploys one service shouldn't hold org-admin rights (lesson 58).

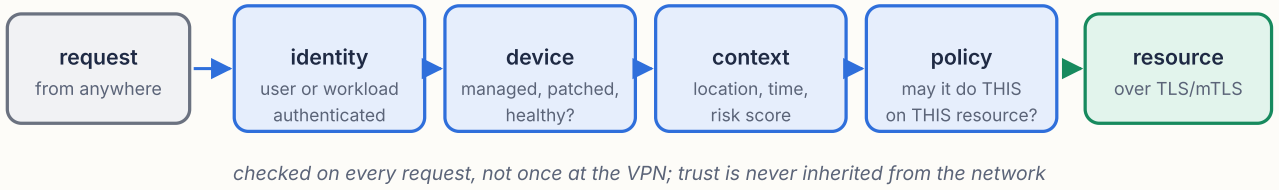
3 Least privilege for people

Standing admin rights are inherited by any stolen laptop or phished session. **Just-in-time access** (Azure PIM, AWS IAM Identity Center with temporary elevation, Teleport, Okta) grants elevated rights on request, for a short time, with approval and an audit trail (lesson 55). Keep a sealed **break-glass** account for emergencies, with alerts on every use.

4 Segment the network

Split the network into zones and allow only needed flows: web → API → database, nothing else crosses. An attacker in one zone can't freely scan the rest. In Kubernetes, start each namespace with a **default-deny NetworkPolicy** and allow specific pod-to-pod traffic; in the cloud, use security groups that reference each other rather than wide CIDR ranges.

5 Zero trust



checked on every request, not once at the VPN; trust is never inherited from the network

Figure 2. Zero trust: every request is authenticated, evaluated in context and authorized for the specific resource.

	Perimeter model	Zero trust (NIST SP 800-207)
Trust from	being inside the network / VPN	verified identity + device + context, per request
Internal traffic	often plain text	encrypted and authenticated (mTLS)
After a breach	attacker moves laterally freely	each hop needs its own authorization
Example	VPN to the office network	Google BeyondCorp: internal apps over the internet, no VPN

WORKED EXAMPLE: PER-SERVICE ROLES ON AN E-COMMERCE PLATFORM

Each microservice runs with its own IAM role. **orders**: read/write on the orders table only. **payments**: read its own secrets and consume its own queue. **email**: `ses:SendEmail` only. Network policies let only the API tier reach the database. When a library flaw gives an attacker code execution in the email service, they can send email, and nothing else; orders data stays out of reach, and the unusual calls show up in the audit trail.

COMMON MISTAKES

- Copying an admin policy “to get it working” and never narrowing it.
- Treating zero trust as a product to buy rather than per-request authentication and authorization.
- Permanent production access for every engineer.
- Flat networks where any pod can reach any database.

6 Key terms

Least privilege: only the permissions needed, only for as long as needed

Just-in-time access: temporary, approved elevation that expires

Zero trust: no implicit trust from network location; verify every request

Blast radius: what an attacker can reach from one compromise

Segmentation: zones with only the required flows allowed

Break-glass account: emergency access, sealed and alerted

7 Interview questions

Q1. What is least privilege? BASIC

Granting each identity only the permissions it needs, for only as long as it needs them.

Q2. What does zero trust mean in practice? BASIC

Never trust a request because of its network location: authenticate and authorize every request, give each service a least-privilege identity, segment the network, encrypt everything (mTLS) and assume breach.

Q3. Engineers need production DB access during incidents. How do you allow it safely? SCENARIO

Just-in-time access: request with a reason, approval, time-boxed credentials (e.g. one hour), session recording or audit logs, and automatic expiry.

Q4. A service's IAM role has s3:* on *. How do you fix it without breaking it? SCENARIO

Use access logs or an access analyzer to see which actions and buckets it really used, generate a narrow policy from that, deploy it, and watch for denials.

Q5. If you have zero trust, do you still need network segmentation? TRICKY

Yes, as defence in depth: segmentation limits what a compromised host can even reach, and covers services that don't yet authenticate every call.

Q6. Does least privilege prevent breaches? TRICKY

Not the initial compromise; it limits what that compromise can do and makes abnormal access stand out.

Next: SD-M-016 · Threat Modeling backendroadmap.com